

S2D-05: Alert Migration - Workflow-Based Alerts

Series: S2D — Splunk to Dynatrace Migration | **Notebook:** 5 of 9 | **Created:** January 2026 | **Last Updated:** 08/11/2026

Overview

While Davis Anomaly Detectors are preferred for continuous monitoring, some Splunk alerts are better suited for workflow-based alerting. This notebook explains when and how to use Dynatrace Workflows for alert migration.

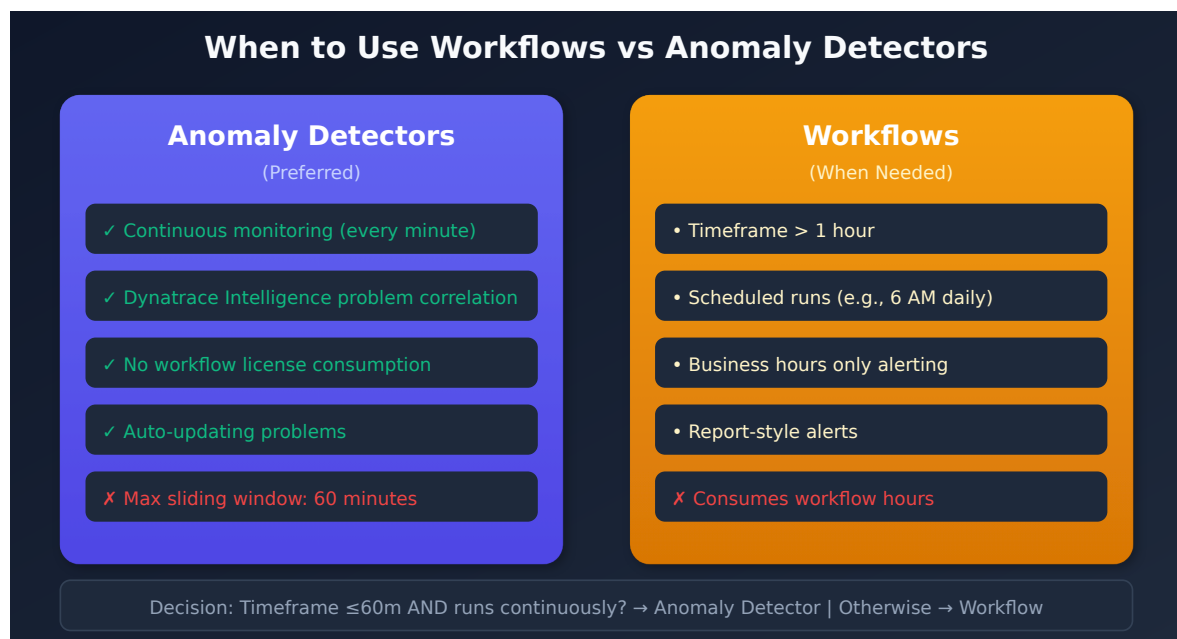


Table of Contents

- [1. When to Use Workflows for Alerting](#)
- [2. Drawbacks of Workflow-Based Alerts](#)

3. [Basic Alerting Workflow Structure](#)
 4. [Example Workflow Query](#)
 5. [Event Creation JavaScript](#)
 6. [Schedule Configuration](#)
 7. [Alternative: Business Hours with Anomaly Detectors](#)
 8. [Important Disclaimers](#)
 9. [Migration Checklist](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Workflows enabled
Permissions	<code>automation.write</code> , <code>logs.read</code>
Knowledge	Understanding of Dynatrace Workflows

Learning Objectives

By the end of this notebook, you will be able to:

1. Identify when workflows are more appropriate than anomaly detectors
2. Structure a basic alerting workflow
3. Configure scheduled triggers
4. Create events from workflow results

When to Use Workflows for Alerting

Use Workflows When:

1. **Query timeframe exceeds 1 hour**
 - Anomaly Detectors have a maximum sliding window of 60 minutes
 - Example: Alert on 7-day log volume trends

2. Alert runs on a schedule (few times per day)

- Scheduled reports that happen at specific times
- Example: Run every weekday at 6 AM

3. Alert is disabled for extended periods

- Business hours-only alerting
- Example: Alert every 5 minutes during 8 AM - 6 PM only

4. Alert is actually a report

- Some Splunk "alerts" are reports that always trigger
- Indicators:
 - Returns multiple appended datasets
 - Threshold of 0 for consistently large results
 - Very large timeframes

Use Anomaly Detectors When:

- Continuous, real-time monitoring is needed
- Timeframe is ≤ 60 minutes
- Alert should run every minute
- Dynatrace Intelligence problem correlation is beneficial

Drawbacks of Workflow-Based Alerts

Before choosing workflows, consider these limitations:

Aspect	Impact
License consumption	Workflows consume workflow hours
Setup effort	More complex than anomaly detectors
Event handling	Manual event creation required
No auto-update	Events won't update or auto-close
No Dynatrace Intelligence correlation	Won't be correlated with other problems

Basic Alerting Workflow Structure

A typical alerting workflow has three main components:

1. Trigger (Schedule)

Defines when the workflow runs:

Schedule Type	Use Case	Example
Fixed time	Daily reports	Every day at 9:00 AM
Time interval	Regular checks	Every 10 minutes
Cron	Complex schedules	Weekdays at 6 AM

2. Query (DQL Execution)

Executes the DQL query to retrieve data:

- Unlike anomaly detectors, does NOT require timeseries output
- Can return single aggregated values
- Timeframe specified in the query, not from notebook/dashboard selector

3. Event Creation (JavaScript)

Evaluates results and creates events if threshold exceeded:

- Extracts values from query results
- Compares against threshold
- Creates custom event with appropriate properties

Example Workflow Query

Unlike Anomaly Detectors, workflow queries often return a single aggregated value:

```
// Workflow alert query - returns single count
// Timeframe explicitly specified (last 7 days)
fetch logs, from:now()-7d
| filter loglevel == "ERROR"
| filter matchesPhrase(k8s.deployment.name, "payment-service")
| summarize error_count = count()
```

Query with Dimensions

Add dimensions to provide context in the event:

```
// Query with dimensions for event context
fetch logs, from:now()-24h
| filter loglevel == "ERROR"
| summarize
    error_count = count(),
    by:{k8s.deployment.name, k8s.namespace.name, dt.entity.cloud_application}
| filter error_count > 100
| sort error_count desc
```

Event Creation JavaScript

The final workflow step uses JavaScript to create events. Here's a template:

```
// Event Creation Template
import { eventsClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function ({ execution_id }) {
  // Get query results from previous step
  const queryResult = await getResult('query_step_name');
  const records = queryResult.records;

  // Configuration
  const THRESHOLD = 100;
  const ALERT_TITLE = '[AppName] High Error Count';

  // Check each result against threshold
  for (const record of records) {
    const errorCount = record.error_count;
    const entityId = record['dt.entity.cloud_application'];

    // No entity, no event. An unattributed event lands on the environment -
    // see below.
    if (errorCount <= THRESHOLD || !entityId) continue;

    await eventsClient.createEvent({
      body: {
        eventType: 'ERROR_EVENT',
        title: ALERT_TITLE,
        // Attribution. This is the line that lets the event join an existing
        // problem.
        entitySelector: `entityId("${entityId}")`,
        properties: {
          'error.count': String(errorCount),
          'deployment.name': record['k8s.deployment.name'],
          'team': 'checkout'
        },
      },
      timeout: 15 // minutes
    });
  }
}
}
```

Why `entitySelector` is the load-bearing line

Davis groups alerts by the entity they are *about*. Most Davis events carry `dt.smartscape_source.id` — the Smartscape entity ID of whatever the signal concerns; 84.9% did on a validation tenant over 7 days, 08/11/2026 — and the documented rule is that the same-Smartscape-entity rule groups all events sharing the same

`dt.smartscape_source.id` value. Events naming the same entity inside the correlation window collapse into **one** problem that updates as the condition persists.

On the ingest API you do not set that field directly. You attribute the event with `entitySelector`, and Davis populates `dt.smartscape_source.id` from the entity it resolves. The failure mode is quiet by design: if `entitySelector` is not set, the event is associated with the environment (`dt.entity.environment`) entity — one bucket for the entire tenant. Technically attributed, useless for correlation.

That fallback is the whole failure, and it runs in the opposite direction from the one people expect. An event with no `entitySelector` does not become unmergeable — it becomes maximally mergeable, because one bucket for the entire tenant means every such alert names the same entity and the correlation rule welds them into the same problem. Measured on a validation tenant over 7 days on 08/11/2026, events that fell back to the environment entity ran at **596 firings per correlation against that single entity** — 28,003 firings in 47 correlations — while events naming a real entity ran at 11. This is a structural failure rather than a sensitivity one, and no threshold change fixes it.

The loop is what makes this destructive. A scheduled workflow that iterates records emits one event per breaching row per run. Attributed, an hourly run against twelve breaching deployments keeps twelve problems updated — one per deployment, each naming the workload that broke. Unattributed, that same run folds all twelve into **one** problem on the environment entity, and every subsequent run folds into it too. You do not get an alert storm; you get a single permanently-open problem that names the whole tenant, cannot be routed to an owner, and no longer tells you which deployment is failing. This is the single most common way a migrated Splunk alert stops being useful in Dynatrace, and it does not look like a threshold problem when you go to debug it.

Getting a usable entity ID

Use an ID the query already returns; do not derive one from a display name.

The dimensioned query in the previous section groups by `dt.entity.cloud_application`, which is a real entity ID (`CLOUD_APPLICATION-0DC6683CE35D5C10`) and drops straight into `entityId(...)`. `k8s.deployment.name` **is not a substitute** — on the tenant this notebook was validated against, those values come back normalized with a trailing wildcard (`"recommendationservice-*"`, `"cartservice-*"`), so matching on the name is not dependable.

Confirm your IDs actually resolve before trusting them. The Smartscape node ID and the classic entity ID are different strings for the same workload — `K8S_DEPLOYMENT-`

184217F70116D1CD and CLOUD_APPLICATION-0DC6683CE35D5C10 — and id_classic is the bridge between them. Run this against your own estate:

```
// Do the entity IDs in your alert query resolve to real workloads?
// A null smartscape_id means that row cannot be attributed - skip it, don't
send it.
fetch logs, from:now()-24h
| filter loglevel == "ERROR"
| summarize error_count = count(), by:{k8s.deployment.name,
k8s.namespace.name, dt.entity.cloud_application}
| filter error_count > 100
| lookup [smartscapeNodes "K8S_DEPLOYMENT" | fields id, id_classic],
        sourceField: dt.entity.cloud_application, lookupField: id_classic, fields:
{smartscape_id = id}
| sort error_count desc
| limit 5
```

Executed against the validation tenant over 24 hours on 07/31/2026, the top five breaching rows resolved like this:

k8s.deployment.name	error_count	resolved smartscape_id
fluent-bit	1,155,207	null
recommendationservice-*	197,539	K8S_DEPLOYMENT-184217F70116D1CD
(null)	145,032	null
emailservice-*	42,587	K8S_DEPLOYMENT-2AE53064B9E3699D
headlessloadgen-*	29,039	K8S_DEPLOYMENT-141CE4BF3FD85A16

Two of the top five do not resolve — including the largest, by two orders of magnitude. The fluent-bit logs carry a dt.entity.cloud_application value with no matching Smartscape deployment node, and the 145,032-error row carries no deployment context at all.

Skip those rows rather than sending an unattributed event for them. An event you cannot attribute is worse than no event: it lands on the environment entity, merges with every other unattributable alert into a problem that names nothing actionable, and no tuning will stop it. If a workload you care about lands in that group, the fix is upstream — get the workload properly monitored — not in the alerting template.

To find detectors already failing this way in your tenant, AIOPS-02 §8 ranks them by firing count and shows which are unattributed; AIOPS-03 §1 covers the correlation rules

themselves, and ALERT-99 §3 explains which Davis data object to count when you audit.

Sources: [Avoid overalerting \(DT docs\)](#), [Ingest an event — POST /api/v2/events/ingest \(DT docs\)](#) — `entitySelector` is optional, and "If not set, the event is associated with the environment (`dt.entity.environment`) entity." **Derived:** the skip-unresolvable-rows rule combines the correlation requirement with the measured resolution gap in the table above.

Schedule Configuration

Fixed Time Schedule

Run at specific times:

```
# Every day at 9:00 AM
schedule:
  type: fixed
  time: "09:00"
  timezone: "America/New_York"
```

Interval Schedule

Run at regular intervals:

```
# Every 6 hours
schedule:
  type: interval
  interval: "6h"
```

Cron Schedule

Complex scheduling with cron expressions:

```
# Weekdays at 6 AM
schedule:
  type: cron
  expression: "0 6 * * 1-5"
  timezone: "America/New_York"
```

Alternative: Business Hours with Anomaly Detectors

For business hours-only alerting, consider these alternatives before using workflows:

Option A: Filter by Hour in Query

Add hour-of-day filtering to exclude off-hours:

```
// Filter logs to business hours only (8 AM - 6 PM)
fetch logs, from:-24h
| filter loglevel == "ERROR"
| fieldsAdd hour = toLong(formatTimestamp(timestamp, format:"HH"))
| filter hour >= 8 and hour < 18
| makeTimeseries count = count(), interval:1m
```

Option B: Workflow to Enable/Disable Detector

Create a workflow that enables/disables the anomaly detector on schedule:

1. Morning workflow (8 AM): Enable detector
2. Evening workflow (6 PM): Disable detector

This preserves the benefits of Dynatrace Intelligence while limiting alert times.

Important Disclaimers

1. Workflows do NOT send notifications directly

- The workflow generates a problem/event
- Notifications are handled by alerting profiles and problem notification workflows

2. Event lifecycle is manual

- Events won't auto-update or auto-close
- Consider event timeout settings

3. License considerations

- Workflow hours are consumed
- See [Workflow consumption documentation](#)

Migration Checklist

Step	Action
1	Confirm workflow is appropriate (see criteria above)
2	Create DQL query with explicit timeframe
3	Confirm the query returns a usable entity ID per row (<code>dt.entity.*</code>), not just a name
4	Configure schedule trigger
5	Implement event creation logic
6	Test workflow execution
7	Configure notification routing

Next Steps

- **S2D-06: ArrayMovingSum** - For extending anomaly detector timeframes
- **S2D-07: Metric Creation** - For performance-critical alerting queries

References

- [Dynatrace Workflows](#)
 - [Workflow Schedules](#)
 - [Workflow Consumption](#)
 - [Event Ingest API](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.